



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Copilot for Power Apps (C803)

Course Code: C803

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	27 July 2026	Initial release — C803 Copilot for Power Apps courseware.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Get Started with Copilot in Power Apps (30%).....	5
Lab 1 — Enable Copilot and Get Oriented in Power Apps Studio.....	6
Lab 2 — Prepare Your Data with Dataverse for Copilot.....	7
Lab 3 — Write Effective Prompts for App Building.....	8
Topic 02 — Build Canvas Apps with Copilot (45%).....	10
Lab 4 — Create a Canvas App from a Prompt.....	10
Lab 5 — Refine Screens, Controls and Layouts with Copilot.....	12
Lab 6 — Generate and Explain Power Fx Formulas with Copilot.....	13
Lab 7 — Add an In-App Copilot Chat for Users.....	14
Topic 03 — Automate and Extend with Copilot (25%).....	15
Lab 8 — Build a Power Automate Flow with Copilot and Call It from the App.....	16
Lab 9 — Automate Approval and Notification Scenarios.....	17
Lab 10 — Share and Govern Your AI-Assisted App.....	19
Wrap-Up.....	20
Next Steps.....	21
Glossary.....	21

Introduction

This Learner Guide accompanies the Copilot for Power Apps (C803) course, conducted by Tertiary Infotech Academy Pte Ltd. It carries the full detail of all 10 hands-on labs, in the order you will run them, together with the concepts each lab depends on.

The labs build one connected result. You take the role of a coordinator at a facilities-management company, 'Harbourfront Facilities', and use Copilot in Power Apps to build a Maintenance Request solution: you enable Copilot and prepare a Dataverse table, generate a canvas app from a prompt, refine its screens and Power Fx, add an in-app Copilot chat, then build an approval-and-notification flow, integrate it with the app, and share and govern it — checking Copilot's work at every step. Wherever you can, use your own non-confidential app idea so you leave with skills applied to your own work; the supplied Harbourfront Facilities scenario is provided for everyone to follow along.

Course Learning Outcomes

- LO1: Enable Copilot in Power Apps and navigate Power Apps Studio ready to build.
- LO2: Prepare a Dataverse table with clear names and descriptions so Copilot can build accurate apps.
- LO3: Write effective prompts that get Copilot to build the app you actually want.
- LO4: Create a working canvas app from a single plain-language prompt.
- LO5: Refine an app's screens, controls and layouts by describing the changes to Copilot.
- LO6: Generate and explain Power Fx formulas with Copilot, and verify them against your data.
- LO7: Add an in-app Copilot chat so end users can query the app's data in natural language.
- LO8: Build a Power Automate flow with Copilot and call it from a canvas app.
- LO9: Automate approval and notification scenarios with Copilot-built flows.
- LO10: Share and govern AI-assisted apps responsibly.

Before You Start — Preparation

What you need

- A laptop (Windows or Mac) with a current Chrome or Edge browser.
- A Microsoft work or school account (not a personal Microsoft account) with access to Power Apps at make.powerapps.com — a Microsoft 365 developer or trial tenant is enough to follow every lab.
- A Power Apps environment with Dataverse and Copilot enabled — the trainer confirms your environment and that Copilot is switched on at the start of the day.
- Access to Power Automate (flow.microsoft.com or the Flows area in Power Apps) using the same account, for the automation labs.

- The 'Harbourfront Facilities — Maintenance' scenario notes (the column list and sample requests the trainer shares) — or your own non-confidential app idea and data.

Verify your setup

Before Lab 1, confirm you can sign in at make.powerapps.com, that you can see and select your Power Apps environment, and that the Copilot pane appears when you create or open an app. If Copilot is not visible or Dataverse is unavailable, tell the trainer.

Sign in at make.powerapps.com · select your environment (top-right) · New > check the Copilot pane appears · open Tables to confirm Dataverse is available

Conventions used in every lab

- Placeholders such as <YOUR ENVIRONMENT> or <YOUR NAME> are replaced with your own values.
- Prompts you give Copilot are shown in a shaded box — type or paste them into the Copilot pane (in Studio) or the app-creation prompt box.
- App paths (for example Power Apps > Solutions, or Studio > Insert > Copilot) and menu names are written as you will use them; Copilot's own buttons and wording may change over time.
- Every lab ends with a 'Test it' step — run the app, formula or flow and verify the result before you move on.

Topic 01 — Get Started with Copilot in Power Apps (30%)

What Copilot in Power Apps and the Power Platform is · Licensing, requirements and enabling Copilot · Copilot in Power Apps Studio · Effective prompting for app building · Preparing data with Dataverse for Copilot

Key concepts

- Copilot in Power Apps — Microsoft's AI assistant built into Power Apps that turns plain-English descriptions into working app screens, tables and formulas.
- The Power Platform — Microsoft's low-code suite (Power Apps, Power Automate, Power BI and Copilot Studio); Copilot spans it, so an app you build can connect to flows and data.
- Canvas apps — the app type you build here: you describe a screen and Copilot lays out the controls, while you keep full control of the design.
- Licensing and requirements — Copilot features need a Power Apps environment with Copilot switched on and a Microsoft work or school account, not a personal one; some tenants also need an admin toggle.
- Enabling Copilot — Copilot is turned on per environment in the Power Platform admin center and in Studio's settings; the trainer confirms it is on before the labs.

- Power Apps Studio — the browser-based design surface where you build the app; the Copilot pane sits alongside the canvas, the tree view and the property pane.
- Effective prompting for apps — a good app-building prompt names the app's purpose, the data it works with, the screens you want and the audience, so Copilot builds the right thing.
- Dataverse — Microsoft's structured cloud database; clear table and column names with descriptions give Copilot the context it needs to build accurate apps and formulas.

Lab 1 — Enable Copilot and Get Oriented in Power Apps Studio

Learning outcome: Sign in to Power Apps, confirm Copilot is enabled, and find your way around Power Apps Studio and the Copilot pane..

Goal: This lab gets Copilot working for you. You sign in at make.powerapps.com, check you are in the right environment, confirm Copilot is switched on, and create a throwaway app so you can see the Copilot pane, the canvas, the tree view and the property pane — the surfaces you use all day. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A confirmed, Copilot-enabled Power Apps environment and a short tour note of where the Copilot pane, tree view, canvas and property pane are. (Tools: make.powerapps.com, Power Apps Studio, Copilot pane, a Microsoft work or school account.)

Step-by-step

1. Open a browser and sign in at make.powerapps.com with your Microsoft work or school account (not a personal Microsoft account). This is the Power Apps maker portal where you build all day.
2. Check your environment: at the top-right, open the environment picker and select the environment your trainer names. Copilot and Dataverse must be enabled there — if the picker is empty or Copilot is missing, tell the trainer.
3. Confirm Copilot is available: from the home page choose 'Start with Copilot' (or '+ Create' > 'Start with a page design'). If a prompt box invites you to 'Describe the app you'd like to build', Copilot is enabled.
4. Create a throwaway canvas app to explore Studio: choose '+ Create' > 'Blank app' > 'Blank canvas app', name it 'Studio Tour', pick Tablet format, and click Create. This opens Power Apps Studio.
5. Find the four surfaces you will use all day, and note where each is: the Tree view (left) listing screens and controls; the Canvas (centre) where the app is designed; the Property pane (right) for the selected control; and the Copilot pane (open it from the Copilot icon on the top toolbar).

6. Ask Copilot what it can do, so you see the pane respond, then read its answer.

What can you help me build and change in this Power Apps canvas app?

7. Confirm the ground rule: Copilot drafts screens, formulas and flows inside Studio, but nothing is shared or published until you choose to. Close the 'Studio Tour' app without saving — it was only for the tour.

Test it

You are signed in to make.powerapps.com in the correct environment, the Copilot pane opens and responds to a question, and you can point to the tree view, canvas, property pane and Copilot pane in Studio.

Note: Full commands and screenshots are in labs/lab-01-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 2 — Prepare Your Data with Dataverse for Copilot

Learning outcome: Create a well-described Dataverse table so Copilot has the context it needs to build an accurate app..

Goal: Copilot builds better apps on well-described data. You create the Maintenance Requests table in Dataverse, add clearly named columns with descriptions and choice options, and add a couple of sample rows — so that when Copilot builds the app in Domain 2 it understands each field. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A Dataverse table named 'Maintenance Requests' with clearly named, described columns, choice options for Category, Priority and Status, and two sample records. (Tools: Dataverse, Tables, columns, choice columns.)

Step-by-step

1. In the maker portal left menu, open 'Tables' (under Dataverse), then choose '+ New table' > 'New table'. Name it 'Maintenance Request' (Power Apps sets the plural 'Maintenance Requests' automatically).
2. Keep the default primary column but rename its display name to 'Title' with the description 'Short summary of the fault' — clear names and descriptions are exactly what Copilot reads to understand your data.

3. Add a single-line text column 'Location' (description: 'Building, floor and room, e.g. Tower A, L3, Server Room').
4. Add a Choice column 'Category' with the choices: Electrical, Plumbing, HVAC, General (description: 'Type of maintenance issue').
5. Add a Choice column 'Priority' with the choices: Low, Medium, High (description: 'How urgent the request is').
6. Add a multiline text column 'Description' (description: 'What is wrong, in the requester's own words').
7. Add a Choice column 'Status' with the choices: New, Approved, In Progress, Resolved (description: 'Where the request is in its lifecycle'), and set its default to New.
8. Add a single-line text column 'Assigned To' (description: 'Technician handling the request') and a multiline text column 'Resolution Notes' (description: 'What was done to fix it'). Save the table.
9. Add two sample rows via 'Edit' (data): e.g. Title 'Flickering light in Tower A lobby', Location 'Tower A, L1, Lobby', Category Electrical, Priority Medium, Status New; and Title 'Server room too warm', Location 'Tower B, L4, Server Room', Category HVAC, Priority High, Status New.

Test it

The 'Maintenance Requests' table exists in Dataverse with named, described columns, Category/Priority/Status have the right choices, and two sample requests are saved.

Note: Full commands and screenshots are in labs/lab-02-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 3 — Write Effective Prompts for App Building

Learning outcome: Compare a vague app prompt with a specific one and capture a reusable four-part prompt pattern for building apps..

Goal: A generated app is only as good as the prompt behind it. You draft a vague prompt, then a specific one that names the app's purpose, the data, the screens and the audience, and distil what worked into a reusable pattern you will use for the rest of the course. You do this in the Copilot prompt box without building the full app yet. **BUILDING BLOCK** — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A written four-part app-prompt pattern (Purpose · Data · Screens · Audience) and one strong, tested app-building prompt saved for reuse. (Tools: Copilot prompt box, prompt design, Purpose/Data/Screens/Audience pattern.)

Step-by-step

1. From the Power Apps home page, open the 'Start with Copilot' prompt box (you will preview prompts here; you build the real app in Lab 4).
2. Type a deliberately vague prompt and read how generic Copilot's proposed app is.

`Build me an app for maintenance.`

3. Now type a specific prompt for the same intent and compare what Copilot proposes.

`Build a canvas app for building staff to log maintenance requests using my Maintenance Requests table in Dataverse. It needs a screen to browse open requests, a screen to view one request in detail, and a form to submit a new request. The audience is non-technical facilities staff, so keep it simple and clearly labelled.`

4. Write down the four parts that made the second prompt work: the Purpose (what the app is for), the Data (which table), the Screens (what pages you want) and the Audience (who uses it).
5. Capture your reusable pattern where you can find it — a notes doc, or pinned in your team's notes.

`PURPOSE: what the app is for | DATA: which Dataverse table | SCREENS: the pages you want | AUDIENCE: who uses it and how simple it must be`

6. Add the conditions that keep results trustworthy: always name the exact table, list the screens explicitly, and state the audience so Copilot pitches the design correctly.
7. Rewrite one app idea of your own using the pattern (or refine the maintenance prompt), and read Copilot's proposal — do not build it yet.
8. Save your best app-building prompt — you will reuse this pattern and this exact prompt in Lab 4 to generate the app.

Test it

You can show two proposals for the same intent (vague vs specific), a written four-part prompt pattern, and one saved app-building prompt ready to use in Lab 4.

Note: Full commands and screenshots are in labs/lab-03-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and

buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Topic 02 — Build Canvas Apps with Copilot (45%)

Creating an app from a prompt · Refining screens, controls and layouts with Copilot · Generating and explaining Power Fx formulas with Copilot · Adding an in-app Copilot chat for users

Key concepts

- Creating an app from a prompt — describe the app you want (its purpose and its data) and Copilot generates a working, multi-screen canvas app you can run straight away.
- Grounding on your data — point Copilot at your Dataverse table so the generated app reads and writes your real records, not placeholder data.
- Refining with Copilot — keep the conversation going: ask Copilot to add a screen, change a layout, sort a gallery or restyle a control, all in plain language.
- Controls and layouts — galleries, forms, labels, buttons and containers are the building blocks Copilot arranges; you then fine-tune them in the property pane.
- Power Fx — Power Apps' Excel-like formula language; Copilot writes Power Fx for you from a description, and explains any formula step by step.
- Reading and trusting formulas — always read the Power Fx Copilot generates, test it against known data, and ask Copilot to explain anything unfamiliar before you rely on it.
- In-app Copilot chat — a Copilot control you can add to an app so your end users can ask questions of the app's data in natural language.
- Human in the loop — Copilot drafts screens, formulas and flows; you run them, test them and decide what to publish.

Lab 4 — Create a Canvas App from a Prompt

Learning outcome: Generate a working, multi-screen canvas app from a single plain-language prompt grounded on your Dataverse table..

Goal: Now you build the real app. You give Copilot the app-building prompt you saved in Lab 3, point it at your Maintenance Requests table, and let it generate a working canvas app with a browse screen, a detail screen and a submit form. You run it, confirm it reads and writes your real data, and save it. **BUILDING BLOCK** — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A working canvas app, 'Harbourfront Maintenance', generated by Copilot and grounded on the Maintenance Requests table, that you can run to browse, view and submit requests. (Tools: Start with Copilot, canvas app generation, Dataverse grounding, Play/Preview.)

Step-by-step

1. From the Power Apps home page choose 'Start with Copilot' and paste the specific app-building prompt you saved in Lab 3.

Build a canvas app for building staff to log maintenance requests using my Maintenance Requests table in Dataverse. It needs a screen to browse open requests, a screen to view one request in detail, and a form to submit a new request. The audience is non-technical facilities staff, so keep it simple and clearly labelled.

2. When Copilot asks which data to use, choose 'Dataverse' and select your 'Maintenance Requests' table so the app is grounded on your real records, not sample data.
3. Review Copilot's plan (the screens and controls it proposes). If it looks right, choose 'Create app' and let Copilot build it — this opens the generated app in Power Apps Studio.
4. Rename the app 'Harbourfront Maintenance' (File > Save, then set the name) and Save it.
5. Run the app with Preview (the play button, or F5). Check the browse screen lists your two sample requests, that selecting one opens its details, and that the new-request form shows your columns.
6. Submit a test request through the form (for example 'Blocked drain in Tower B washroom', Plumbing, Low), then confirm it appears in the browse gallery — proving the app writes back to Dataverse.
7. Verify in the data: open the Maintenance Requests table's data view in another tab and confirm your test request is stored there.
8. Close Preview and Save. You now have a working, data-connected app to refine in the next labs.

Test it

The 'Harbourfront Maintenance' app runs, its browse screen lists real records from Dataverse, and a request submitted through the form appears both in the app and in the Maintenance Requests table.

Note: Full commands and screenshots are in labs/lab-04-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 5 — Refine Screens, Controls and Layouts with Copilot

Learning outcome: Improve the generated app's screens, controls and layout by describing the changes to Copilot in plain language..

Goal: A generated app is a starting point. You keep the conversation going with Copilot to make the app clearer and more useful: sort and filter the gallery, relabel and reorder fields, tidy the layout and adjust styling — all by describing what you want rather than editing by hand. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A refined 'Harbourfront Maintenance' app: the browse gallery sorted and filtered sensibly, clearer labels, and a tidier layout produced by describing changes to Copilot. (Tools: Copilot pane in Studio, galleries, forms, labels, layout and styling.)

Step-by-step

1. Open the 'Harbourfront Maintenance' app in Studio and open the Copilot pane.
2. Ask Copilot to sort the browse gallery so the newest requests are first.

`Sort the requests gallery on the browse screen so the most recently created requests appear at the top.`

3. Ask Copilot to show only open work by filtering out resolved requests.

`On the browse screen, show only requests whose Status is not Resolved.`

4. Ask Copilot to make each gallery item clearer by showing the key fields.

`In each item of the requests gallery, show the Title in bold, then Location and Category on one line, and show the Priority as a coloured tag.`

5. Ask Copilot to improve a label and the screen title for a non-technical audience.

`Rename the browse screen's title to 'Open Maintenance Requests' and change the new-request button label to '+ New Request'.`

6. Ask Copilot to tidy the detail screen's layout.

`On the detail screen, group the request fields into a single readable card with clear field labels and consistent spacing.`

7. After each change, run Preview (F5) and confirm the app still works and looks clearer; if a change is not what you meant, tell Copilot what to adjust and re-run.
8. Save the app. Keep note of one change Copilot made that you would otherwise have done by hand — that is the time it saves you.

Test it

The browse gallery is sorted newest-first and hides resolved requests, gallery items and the detail screen are clearer, and the app still runs correctly in Preview after the refinements.

Note: Full commands and screenshots are in labs/lab-05-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 6 — Generate and Explain Power Fx Formulas with Copilot

Learning outcome: Use Copilot to write Power Fx formulas from a description, explain existing formulas, and verify them against your data..

Goal: Power Fx is what makes the app smart. You ask Copilot to write Power Fx from a plain description — a priority colour, a count of open requests, a days-open calculation — paste each formula onto the right control, then ask Copilot to explain a formula step by step and verify every result against your data. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

Several verified Power Fx formulas on your app (a priority colour, an open-requests count, and a days-open value), plus a plain-English explanation of one formula. (Tools: Power Fx, Copilot formula generation and explanation, verification.)

Step-by-step

1. With the app open, select the Priority tag/label in the gallery. Ask Copilot for a formula that colours it by priority.

Write a Power Fx formula for the Fill (or Color) of this label that shows red when Priority is High, amber when Medium, and green when Low.

2. Paste Copilot's formula onto the control's Color/Fill property. Below is the shape to expect — adjust the field and choice names to match your table.

```
Switch(ThisItem.Priority.Value, "High", Color.Red, "Medium", Color.Orange, "Low", Color.Green, Color.Gray)
```

3. Verify: run Preview and confirm a High request shows red, a Medium amber, a Low green. If a colour is wrong, tell Copilot which value is off and re-run.
4. Add a header count of open requests. Select the browse screen header, add a label, and ask Copilot for the formula.

Write a Power Fx formula for a label's Text that counts how many requests in the Maintenance Requests table have a Status that is not Resolved, shown as 'Open requests: N'.

5. Paste the formula and check the number matches what you can count in the gallery. Expected shape:

```
"Open requests: " & CountRows(Filter('Maintenance Requests', Status.Value <> "Resolved"))
```

6. On the detail screen, add a 'Days open' label and ask Copilot for a formula using the created date.

Write a Power Fx formula for a label's Text that shows how many whole days it has been since this request was created, using its Created On date.

7. Ask Copilot to explain one of its formulas so you understand it before you rely on it.

Explain, step by step, what this formula does: `Switch(ThisItem.Priority.Value, "High", Color.Red, "Medium", Color.Orange, "Low", Color.Green, Color.Gray)`

8. Read the explanation and write one sentence in your notes on what Switch does and why you would trust this formula. Save the app.

Test it

The priority tag is colour-coded correctly, the header shows a correct count of open requests, the detail screen shows a sensible days-open value, and you can explain in one sentence what one of the formulas does.

Note: Full commands and screenshots are in labs/lab-06-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 7 — Add an In-App Copilot Chat for Users

Learning outcome: Add a Copilot chat control so end users can ask questions of the app's data in natural language..

Goal: Copilot does not only help you build — it can help your users too. You add the in-app Copilot control to a screen, point it at your Maintenance Requests data, write a couple of example questions users might ask, and test that the chat answers from the app's real records. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

An in-app Copilot chat on the 'Harbourfront Maintenance' app that answers end-user questions from the Maintenance Requests data. (Tools: Copilot control (Insert > Copilot), grounding on Dataverse, end-user Q&A.)

Step-by-step

1. Confirm the Copilot control is available in your environment (an admin setting enables the in-app Copilot control). If it is off, the trainer will show a shared app with it enabled.
2. Open the app in Studio, go to the browse screen, then Insert > Input (or 'Copilot') and add the 'Copilot' control to the screen.
3. Set the control's data source to your 'Maintenance Requests' table so it answers from your app's records, not the open web.
4. Position and size the control so it sits neatly on the screen (for example a panel on the right), and give the screen room for it.
5. Run Preview and ask the in-app Copilot a question a real user would ask.

How many high-priority requests are still open, and where are they?

6. Ask a second, more specific question and confirm the answer matches what you can see in the gallery.

List the requests in Tower A that are not yet resolved.

7. Verify: cross-check one of Copilot's answers against the browse gallery or the table data — never present an answer you have not confirmed.
8. Save and publish a first version of the app (File > Save, then Publish) so it is ready to automate in Domain 3.

Test it

The in-app Copilot control answers at least two natural-language questions from your Maintenance Requests data, and its answers match what you can verify in the app, with a first version published.

Note: Full commands and screenshots are in labs/lab-07-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Topic 03 — Automate and Extend with Copilot (25%)

Building Power Automate flows with Copilot · Integrating flows with canvas apps · Approval and notification scenarios · Sharing and governing AI-assisted apps

Key concepts

- Power Automate — the Power Platform's workflow tool; Copilot builds automated flows from a plain-language description of the trigger and the actions.
- Building flows with Copilot — describe what should happen and when (for example 'when a high-priority request is created, email the manager for approval') and Copilot assembles the flow.
- Integrating flows with canvas apps — a flow can be called from a canvas app with Power Fx, so a button in your app starts the automation.
- Approvals — Power Automate's Approvals action sends a request to an approver, waits for their decision, and branches the flow on approve or reject.
- Notifications — flows can send email, Teams messages or mobile notifications to keep people informed as records change.
- Sharing apps — canvas apps and their flows are shared with named users or groups, as run-only users or co-owners.
- Governing AI-assisted apps — environments, data loss prevention (DLP) policies and connector controls keep low-code apps safe; the owner stays accountable for what the app does.
- Verify and document — test every Copilot-built flow end to end, and record what the app and flow do so the solution stays maintainable.

Lab 8 — Build a Power Automate Flow with Copilot and Call It from the App

Learning outcome: Use Copilot to build an automated flow, then integrate it with the canvas app so a button starts the automation..

Goal: Now you automate. Using Copilot in Power Automate, you describe a flow that runs when a high-priority request is created and emails the facilities manager. You test it, then wire it into your canvas app so submitting a request can trigger the automation — connecting the app you built to a real workflow. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A Copilot-built Power Automate flow that notifies the manager of high-priority requests, tested and callable from the 'Harbourfront Maintenance' app. (Tools: Power Automate, Copilot flow builder, Dataverse trigger, Office 365 Outlook, Power Fx integration.)

Step-by-step

1. Go to Power Automate (from the app menu 'Flows', or make.powerautomate.com) in the same environment, and choose to create a flow with Copilot — the 'Describe it to design it' prompt box.
2. Describe the flow you want in plain language and let Copilot assemble it.

When a new row is created in my Maintenance Requests table in Dataverse and its Priority is High, send an email to the facilities manager with the request's Title, Location and Description.

3. Review the flow Copilot built: confirm the trigger is 'When a row is added' on Maintenance Requests, that there is a condition on Priority = High, and that the action is 'Send an email (V2)'. Add the manager's address (use your own for testing).
4. Save the flow, name it 'Notify manager of high-priority request', and test it: create a High-priority request (in the app or the table) and confirm the email arrives.
5. Read what Copilot built, step by step, so you understand it — ask Copilot to explain any action you are unsure of before you rely on the flow.
6. Integrate the flow with the app: open 'Harbourfront Maintenance' in Studio, add the Power Automate flow to the app (Power Automate pane > add your flow), and select the submit button.
7. Ask Copilot in Studio for the Power Fx to run the flow when the button is pressed, or add it yourself. Expected shape (name matches your flow):

```
'Notifymanagerofhighpriorityrequest'.Run(TitleInput.Text, LocationInput.Text, DescriptionInput.Text)
```

8. Run Preview, submit a High-priority request, and confirm both that the record is saved and that the flow runs and the email arrives. Save the app.

Test it

The flow runs when a high-priority request is created (or the app button is pressed), the manager receives an email with the request details, and you can explain what each step of the flow does.

Note: Full commands and screenshots are in labs/lab-08-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 9 — Automate Approval and Notification Scenarios

Learning outcome: Extend the flow into an approval-and-notification scenario that updates the request based on the decision..

Goal: A notification is good; a decision is better. You extend the flow with Copilot so a high-priority request is sent to the manager for approval, and when they approve or reject, the flow updates the request's Status and notifies the requester of the outcome — a complete approval-and-notification loop. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

An approval-and-notification flow: high-priority requests go to the manager for approval, the request Status is updated on the decision, and the requester is notified of the outcome. (Tools: Power Automate Approvals, conditions/branches, Dataverse update row, notifications.)

Step-by-step

1. Open your 'Notify manager of high-priority request' flow and edit it. Ask Copilot to turn the notification into an approval.

Instead of just emailing the manager, start an approval and wait for the manager to approve or reject this high-priority request.

2. Confirm Copilot added a 'Start and wait for an approval' action (Approve/Reject type) addressed to the manager, with the request's details in the approval message.
3. Add the decision branch. Ask Copilot to handle both outcomes.

After the approval, if it is approved set this request's Status to Approved; if it is rejected set the Status to Resolved with a note that it was rejected.

4. Confirm the flow now has a Condition on the approval outcome and an 'Update a row' action on Maintenance Requests in each branch that sets Status correctly.
5. Add the requester notification. Ask Copilot to close the loop.

After the status is updated, send an email to the person who created the request telling them whether it was approved or rejected.

6. Save the flow and test the approve path: create a High-priority request, open the approval (in Power Automate, Outlook or Teams), approve it, and confirm the request's Status becomes Approved and the requester is emailed.
7. Test the reject path with a second request: reject it, and confirm the Status and the requester's email reflect the rejection.
8. Check the app: open 'Harbourfront Maintenance' and confirm the browse gallery now shows the updated statuses from the flow — the automation and the app are working together.

Test it

Approving a high-priority request sets its Status to Approved and emails the requester; rejecting one sets the Status accordingly and emails the requester; and the app's gallery reflects both outcomes.

Note: Full commands and screenshots are in labs/lab-09-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Lab 10 — Share and Govern Your AI-Assisted App

Learning outcome: Share the app and flow with the right people and apply sensible governance to your AI-assisted solution..

Goal: An app only helps if the right people can use it — safely. You share the app with users and the flow's connections, review who can do what, and apply governance: the environment it lives in, data loss prevention awareness, and documenting what the solution does so it stays maintainable and accountable. BUILDING BLOCK — what you do in this lab becomes part of your Harbourfront Facilities Maintenance Request solution, the single deliverable you build, refine, automate and govern across all 10 labs.

What you'll build

A shared 'Harbourfront Maintenance' app with the flow's connections in place, plus a short governance note covering sharing, environment, DLP and documentation. (Tools: App sharing, run-only vs co-owner, environments, DLP awareness, documentation.)

Step-by-step

1. In the maker portal, open 'Apps', select 'Harbourfront Maintenance', and choose 'Share'.
2. Share with a colleague or a group as run-only users (they can use the app but not edit it), and note the difference between run-only users and co-owners who can edit. Confirm the app's flow connections are included so the automation still works for shared users.
3. Review governance — the environment: confirm which environment the app and flow live in, and understand that moving between Dev/Test/Prod environments is how organisations control release. Note your environment name.
4. Review governance — data loss prevention (DLP): understand that admins use DLP policies to control which connectors (Dataverse, Outlook, Teams, etc.) can be combined; check your app only uses connectors your organisation allows.
5. Ask Copilot to help you document the solution so it stays maintainable.

Write a short description of this app and its flow for other makers: what it does, which Dataverse table it uses, what the approval flow does, and who it is shared with.

6. Save that description in the app's details (Apps > ... > Details > Description) and in your team notes, so the next person understands the solution.
7. Confirm accountability: you — the owner — are responsible for what the app and flow do; Copilot only helped build them. Note who to contact if the flow needs changing.

8. Final check: run the shared app once more end to end (submit a High-priority request, approve it, see the status update) to confirm the complete Harbourfront Facilities solution works.

Test it

The app is shared with at least one user or group with the flow's connections included, you can state the environment and which connectors the app uses, and a plain-language description of the solution is saved with the app.

Note: Full commands and screenshots are in labs/lab-10-*.md. Use only accounts and data you are authorised to use. Never put passwords, personal identifiers or confidential business data into a Copilot prompt or a Dataverse table used for practice — use the supplied Harbourfront Facilities sample scenario if in doubt. Power Apps, Copilot and Power Automate screens, menu names and buttons may differ slightly between tenants and change over time; the trainer will point out the current location on the day.

Wrap-Up

In one day you have taken a facilities team's everyday need — logging, approving and tracking maintenance requests — and turned it into a working, automated Power Apps solution, using Copilot at every step and checking its work before publishing.

What you built

- Copilot enabled in Power Apps Studio and a clear picture of what Copilot can and cannot do.
- A well-described Dataverse table that gives Copilot the context to build accurately.
- A reusable app-prompt pattern (purpose, data, screens, audience) you can apply to any app.
- A working canvas app generated from a prompt, with refined screens, controls and layouts.
- Power Fx formulas written and explained by Copilot — and verified by you against known data.
- An in-app Copilot chat that lets end users ask questions of the app's data.
- A Power Automate approval-and-notification flow, built with Copilot and called from the app.
- The app and flow shared with the right people and governed with sensible safeguards.

What to do next

- Point these techniques at one real, recurring request or process in your own team and build it.

- Keep verifying: run every Copilot-built app and flow end to end, and read the Power Fx before you trust it.
- Save your best app-building prompts so you and your team can reuse them.
- Keep confidential data out of prompts and practice tables, and note where Copilot helped so your work stays accountable.

Next Steps

- First pass: complete every lab yourself, following the steps and verifying each 'Test it' check.
- Second pass: rebuild the Maintenance Request solution from memory, writing your own Copilot prompts.
- Apply the techniques to a real, non-confidential app or process from your own organisation.
- Review each lab's detailed steps in this guide and re-run the labs in your own Power Apps environment.

Glossary

- **Copilot** — Microsoft's AI assistant built into Power Apps; it turns plain-English descriptions into app screens, Power Fx formulas and flows, and explains what it built.
- **Power Apps** — Microsoft's low-code platform for building business applications; this course builds canvas apps in it.
- **Power Platform** — Microsoft's low-code suite — Power Apps, Power Automate, Power BI and Copilot Studio — that share data and connectors.
- **Canvas app** — A Power Apps app where you (and Copilot) arrange controls freely on each screen, giving full control of the layout.
- **Power Apps Studio** — The browser-based design surface at make.powerapps.com where you build and edit an app, with the Copilot pane alongside.
- **Dataverse** — Microsoft's structured cloud database of tables, columns and relationships that stores the app's data.
- **Table** — A Dataverse structure that holds records (rows) with defined columns — for example the Maintenance Requests table in this course.
- **Column** — A single field in a table (such as Priority or Status); clear names and descriptions help Copilot build accurately.
- **Prompt** — The plain-language instruction you give Copilot; a good app prompt states the purpose, the data, the screens and the audience.
- **Power Fx** — Power Apps' Excel-like formula language; Copilot can write it from a description and explain any formula step by step.

- **Gallery** — A control that lists many records (rows) from a table, such as the list of maintenance requests.
- **Form** — A control for viewing, editing or creating a single record, such as submitting a new request.
- **Power Automate** — The Power Platform's workflow tool; Copilot builds automated flows from a description of the trigger and actions.
- **Flow** — An automated workflow in Power Automate — a trigger followed by actions, such as sending an approval when a request is created.
- **Approval** — A Power Automate action that sends a request to an approver, waits for their decision, and branches on approve or reject.
- **Connector** — A pre-built link that lets an app or flow talk to a service such as Dataverse, Outlook or Teams.
- **Data loss prevention (DLP)** — Governance policies that control which connectors an app or flow may combine, to keep business data safe.
- **Publish** — Saving and releasing a version of an app so shared users run the latest version.
- **Human in the loop** — The practice of a person reviewing, testing and approving AI-generated screens, formulas and flows before they are relied upon.